

Python + PostgreSQL

What's changed since your 2022 course — a practical update guide (as of mid-2026)

Your course teaches Python 3.11 and PostgreSQL basics from 2022. The core language and SQL you're learning is still completely valid — nothing you learn will be wasted. But the library most 2022-era courses use to connect Python to PostgreSQL (**psycopg2**) has a modern successor, and PostgreSQL itself has moved from version 14 to version 18 with several useful new features. This guide highlights exactly what to swap in.

1. The big one: psycopg2 → psycopg (a.k.a. psycopg3)

Most 2022 courses teach **psycopg2**. Its successor, simply called **psycopg** (often referred to as “psycopg3”), has been the actively developed, recommended choice for new projects since 2021–2022, and by now it's the mainstream default. psycopg2 still works and isn't deprecated, but it receives maintenance only — no new features. If you're starting fresh, learn psycopg3.

Installation

Old (2022 course):

```
pip install psycopg2-binary
```

Now:

```
pip install "psycopg[binary]"
```

for the newer connection pool:

```
pip install "psycopg[pool]"
```

Basic connection

Old:

```
import psycopg2

conn = psycopg2.connect(
    host="localhost", dbname="mydb",
    user="postgres", password="secret"
)
cur = conn.cursor()
cur.execute("SELECT * FROM employees WHERE id = %s", (5,))
row = cur.fetchone()
conn.commit()
cur.close()
conn.close()
```

Now:

```
import psycopg

# psycopg3 connections are context managers -- they auto-commit
# on clean exit and auto-rollback on exception, and auto-close.
with psycopg.connect(
    "host=localhost dbname=mydb user=postgres password=secret"
) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM employees WHERE id = %s", (5,))
```

```
        row = cur.fetchone()
# no manual .commit() / .close() needed
```

Note: psycopg3 also accepts the same keyword style you already know (`psycopg.connect(host=..., dbname=..., user=..., password=...)`), so nothing you learn about connection parameters is wasted — only the import name and the recommended context-manager usage change.

Connection has `.execute()` directly — a genuinely new shortcut

In psycopg3 you often don't need to create a cursor object explicitly for simple queries:

```
with psycopg.connect(conninfo) as conn:
    for row in conn.execute("SELECT id, name FROM employees"):
        print(row)
```

Getting dict-like rows (instead of `psycopg2.extras.RealDictCursor`)

Old:

```
import psycopg2.extras
cur = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
cur.execute("SELECT * FROM employees")
row = cur.fetchone()
print(row["name"])
```

Now:

```
from psycopg.rows import dict_row

with psycopg.connect(conninfo, row_factory=dict_row) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM employees")
        row = cur.fetchone()
        print(row["name"])

# other built-in row factories: tuple_row (default), namedtuple_row,
# class_row(SomeDataclass) -- maps rows straight into your own classes
```

Native async support (no separate library needed)

psycopg2 needed extensions/other libraries for async. psycopg3 supports asyncio natively:

```
import asyncio, psycopg

async def main():
    async with await psycopg.AsyncConnection.connect(conninfo) as conn:
        async with conn.cursor() as cur:
            await cur.execute("SELECT * FROM employees")
            rows = await cur.fetchall()

asyncio.run(main())
```

Connection pooling

Old:

```
from psycopg2 import pool
p = pool.SimpleConnectionPool(1, 10, dsn=conninfo)
```

Now (separate installable package, actively maintained):

```
from psycopg_pool import ConnectionPool
```

```
pool = ConnectionPool(conninfo, min_size=1, max_size=10)
with pool.connection() as conn:
    conn.execute("SELECT 1")
```

A couple of gotchas when porting old code

- **IN clauses:** psycopg2 let you pass a tuple directly for `WHERE id IN %s`. psycopg3 doesn't — use `WHERE id = ANY(%s)` with a list instead.
- **Client-side vs server-side binding:** psycopg3 sends parameters to the server separately from the query by default (more secure, closer to how PostgreSQL itself works) rather than doing client-side string substitution like psycopg2.
- Percent-style placeholders (`%s` and `%(name)s`) still work exactly as before — that part of your course material carries over directly.

2. PostgreSQL itself: v14 (2022) → v18 (2026)

PostgreSQL ships a new major version every year. Your course likely assumes v14. As of mid-2026, the current stable release is **PostgreSQL 18** (with 19 in beta). None of your SQL knowledge is invalidated — these are additive features you can use once you know they exist.

Version	Year	Notable additions relevant to a learner
15	2022	MERGE statement (upsert-style conditional INSERT/UPDATE/DELETE in one command)
16	2023	Faster logical replication; more granular built-in roles for permissions
17	2024	SQL/JSON standard functions: JSON_TABLE(), JSON_QUERY(), JSON_EXISTS(); incremental backups; MERGE .
18	2025	Asynchronous I/O for faster reads; virtual generated columns; native UUIDv7 support (time-sortable UUIDs)

MERGE — new way to do "upsert" or conditional writes

Old-style upsert your course probably teaches (still fully valid):

```
INSERT INTO employees (id, name, salary)
VALUES (5, 'Alice', 70000)
ON CONFLICT (id) DO UPDATE
SET salary = EXCLUDED.salary;
```

New option (PostgreSQL 15+), better for multi-branch logic:

```
MERGE INTO employees AS e
USING (VALUES (5, 'Alice', 70000)) AS src(id, name, salary)
ON e.id = src.id
WHEN MATCHED THEN
    UPDATE SET salary = src.salary
WHEN NOT MATCHED THEN
    INSERT (id, name, salary) VALUES (src.id, src.name, src.salary);
```

ON CONFLICT is still the right tool for simple upserts; reach for MERGE when you need several different conditional actions in one statement.

Identity columns instead of SERIAL

SERIAL predates 2022 too, but many 2022-era courses still teach it as the go-to for auto-incrementing primary keys. The SQL-standard, currently recommended form is GENERATED ... AS IDENTITY:

Old:

```
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name TEXT
);
```

Now:

```
CREATE TABLE employees (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT
);
```

Why: SERIAL is really sugar around a sequence + default + ownership, with quirks around permissions and dumps/restores. IDENTITY is the ANSI SQL standard way and avoids those quirks.

3. If the course also touches an ORM (SQLAlchemy)

If a later section uses SQLAlchemy, be aware SQLAlchemy 2.0 (released Jan 2023) was a deliberate, significant rewrite of the API. 2022 material almost always teaches "1.x style." Both still run, but 2.0-style is what you'll see in current docs and job code.

Old (1.x style):

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine("postgresql+psycopg2://user:pw@localhost/mydb")
Session = sessionmaker(bind=engine)
session = Session()

result = session.query(Employee).filter_by(id=5).first()
```

Now (2.0 style):

```
from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session

# psycopg3 dialect string is just "postgresql+psycopg://"
engine = create_engine("postgresql+psycopg://user:pw@localhost/mydb")

with Session(engine) as session:
    stmt = select(Employee).where(Employee.id == 5)
    result = session.scalars(stmt).first()
```

Model declaration also changed, from Column()-based classes to typed Mapped[] annotations:

```
# Old
class Employee(Base):
    __tablename__ = "employees"
    id = Column(Integer, primary_key=True)
    name = Column(String)

# New (2.0 style, uses Python type hints)
class Employee(Base):
    __tablename__ = "employees"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
```

4. Minor Python 3.11 → 3.13 changes worth knowing

Nothing here breaks your course material — these are just conveniences that showed up after 3.11 that you'll see in modern example code:

- **3.11** (your course's version): exception groups + `except*`, `tomllib` for reading TOML config, much better tracebacks pointing at the exact sub-expression that failed.
- **3.12**: new, more flexible f-string parser (you can reuse quote characters and nest expressions more freely); type parameter syntax for generics (`def first[T](l: list[T]) -> T`).
- **3.13**: improved, colorized interactive REPL; experimental free-threaded ("no-GIL") build; better error messages continue to improve.

None of these change how you talk to PostgreSQL — they're general-purpose language quality-of-life improvements.

5. Quick-reference cheat sheet

Doing this?	2022-style	2026-recommended
Install driver	<code>pip install psycopg2-binary</code>	<code>pip install "psycopg[binary]"</code>
Import	<code>import psycopg2</code>	<code>import psycopg</code>
Connect	<code>psycopg2.connect(...)</code>	<code>psycopg.connect(...)</code> as context manager
Dict rows	<code>psycopg2.extras.RealDictCursor</code>	<code>row_factory=dict_row</code>
Async	3rd-party add-ons	built in: <code>AsyncConnection</code>
Pooling	<code>psycopg2.pool</code>	<code>psycopg_pool.ConnectionPool</code>
Upsert	<code>INSERT ... ON CONFLICT</code>	<code>ON CONFLICT</code> (simple) or <code>MERGE</code> (complex)
Auto-increment PK	<code>SERIAL</code>	<code>GENERATED ALWAYS AS IDENTITY</code>
SQLAlchemy query	<code>session.query(Model)</code>	<code>session.scalars(select(Model))</code>

Bottom line: follow the course as written to learn the fundamentals (connections, cursors, transactions, parameterized queries, CRUD) — that logic is 100% unchanged. Swap `psycopg2` for `psycopg` as you type the code, use context managers instead of manual `commit/close`, and keep this sheet nearby for the SQL-side extras (`MERGE`, `IDENTITY` columns, new JSON functions) once you're comfortable with the basics.